

第5章: Supabaseのセットアップ

アプリで登録した本のデータは、どこかに保存しておく必要があります。この章では、データの保管庫 = バックエンド／データベースを用意します。まず「どの保存方法を選ぶべきか」を比較したうえで、本書が採用する Supabase をセットアップし、アプリから接続できるようにします。

1. そもそも「データを保存する」とは

第1章で作ったアプリは、一度閉じると入力した内容が消えてしまいます。本のリストをずっと残しておくには、データを「永続化（えいぞくか）」する場所が必要です。

「永続化（persistence）」とは？「アプリを閉じて、スマホを再起動しても、データが消えずに残ること」。逆に、アプリの実行中だけ覚えている（閉じると消える）状態を「一時的（揮発性）」と呼びます。state は一時的、データベースは永続的です。

データの保存先には、大きく分けて2つの場所があります。

場所	特徴	例
端末内（ローカル）	そのスマホの中だけに保存。ネット不要だが、別の端末とは共有できない	SQLite、AsyncStorage
クラウド（サーバー）	インターネットの向こうのサーバーに保存。複数端末で同じデータを共有できる	Supabase、Firebase

本書では、複数端末で同じ本棚を見られるよう、クラウド型の Supabase を使います。

2. データ保存方法の選択肢を比較する

「どんな場合にどれを選ぶべきか」を、第0章よりさらに詳しく解説します。アプリの目的によって最適な選択は変わります。

2.1 主要な選択肢の比較表

選択肢	種類	置き場所	複数端末 同期	オフ ライン	SQL	向いているアプリ
Supabase（本書採用）	RDB （PostgreSQL）	クラウド	◎	△	○	SNS、共有する本棚、複数 端末で使うもの
Firebase （Firestore）	NoSQL	クラウド	◎	◎	×	チャット、リアルタイム共同編 集
expo-sqlite	RDB（SQLite）	端末内	×	◎	○	完全オフラインのメモ・家計 簿
AsyncStorage	キー・バリュー	端末内	×	◎	×	設定値・小さなデータの保存
WatermelonDB	RDB（SQLiteベ ース）	端末内＋ 同期	○	◎	△	大量データ＋オフライン重視

2.2 それぞれの解説

◆ Supabase（本書の採用）

Supabase（スーパベース）は、PostgreSQL（ポストグレスキューエル、業界標準のRDB）をベースにした「BaaS（Backend as a Service）」です。

「BaaS」とは？ Backend as a Service の略。「バックエンド（サーバー側の機能）を、自分で作らずサービスとして借りる」という意味。データベース・API・認証などを、設定するだけで使えます。

- **長所:** SQL（業界標準のデータ操作言語）が使える／複数端末で同期できる／無料枠が十分／本書のWeb版（Next.js）とまったく同じバックエンドを共用できる／オープンソース
- **短所:** 完全オフライン動作には工夫が必要

◆ Firebase（Firestore）

Googleが提供するBaaS。データベースは **Firestore** という **NoSQL**（表形式でない）型。

- **長所:** リアルタイム同期やオフライン対応が得意／Googleサービスと連携しやすい
- **短所:** SQLが使えず独自の操作方法を覚える必要がある／他サービスへの移行がしにくい

◆ expo-sqlite（端末内DB）

スマホの中に **SQLite**（軽量のRDB）を持つ方式。

- **長所:** ネット完全不要／高速／サーバー費用ゼロ
- **短所:** そのスマホの中だけ。機種変更やPC・別端末とは共有できない

◆ AsyncStorage

「キーと値」だけの最もシンプルな端末内保存。設定値などの小さなデータ向け。

どんな時にどれを選ぶ？（まとめ） - 複数端末で同期したい／Webアプリとデータを共有したい → Supabase（本書） - チャットなどリアルタイム性が最優先 → Firebase - ネット不要の完全オフライン・自分専用アプリ → expo-sqlite - テーマ設定やログイン状態など小さな値だけ → AsyncStorage（他のDBと併用も多い）

3. Supabaseのアカウント作成とプロジェクト作成

Web版チュートリアルを既にやった方へ: 同じSupabaseプロジェクトを使い回せます。その場合は「3.4 既存テーブルの確認」へ進み、**books** テーブルがあれば新規作成は不要です。

3.1 アカウント作成

1. ブラウザで <https://supabase.com/> を開きます。
2. 「Start your project」または「Sign In」をクリック。
3. GitHubアカウントでサインアップするのが簡単です（第1章でGitHubを使う前提）。メールアドレスでの登録も可能です。

3.2 新しいプロジェクトを作る

1. ダッシュボード（管理画面）で「New project」をクリック。
2. 以下を入力します。

項目	入力内容
Name	プロジェクト名（例: books-app ）。自分が分かる名前でもOK
Database Password	データベースのパスワード。 必ず控えておく （後で必要）
Region	サーバーの場所。日本なら「 Northeast Asia (Tokyo) 」が高速

3. 「Create new project」を押すと、数分でデータベースが用意されます。

「Region（リージョン）」を東京にする理由: サーバーが物理的に近いほど、データのやり取りが速くなります。日本のユーザー向けアプリなら東京を選ぶと体感速度が上がります。

3.3 テーブルを作る — **books** テーブル

データベースの中に「本の情報を入れる表（テーブル）」を作ります。Supabaseの **SQL Editor**（SQLを書いて実行する画面）を使うのが確実です。

1. Supabaseの左メニューから「**SQL Editor**」を開きます。
2. 次のSQLを貼り付けて、右下の「**Run**」を押します。

▼このコードがやること（先に日本語で）：データベースの中に「本の情報を保存する表（テーブル）」を新しく作ります。SQLとは「データベースに命令を出すための言語」で、**create table** は「表を作りなさい」という命令です。表には「どんな列（タイトル・著者・状態など）を持つか」「各列はどんな種類の値か」をあらかじめ決めて書きます。まずは「表 = Excelのシートのようなもの」「列 = シートの縦の項目」とイメージできればOKです。

```
-- create table : 新しいテーブル（表）を作るSQL命令
-- books          : テーブル名（本のデータを入れる表）
create table books (
  -- id : 各行を区別する識別番号。uuid型で、自動生成される
  id uuid primary key default gen_random_uuid(),
  -- primary key      : 「主キー」。各行を一意に識別する特別な列
  -- default gen_random_uuid() : 値を省略したら自動でランダムなIDを振る

  title text not null,          -- title : 本のタイトル。text型（文字列） / not null = 空は許さない（必須）
  author text not null,         -- author : 著者名。必須
  status text not null default '未読', -- status : 読書状態。省略時は '未読' になる (default)
  memo text,                   -- memo : メモ。not nullが無いので空でもOK（省略可能）

  -- created_at : 登録日時。timestamp型で、登録時の時刻が自動で入る
  created_at timestamp with time zone default now()
  -- default now() : 省略時は「今の時刻」を自動で入れる
);
```

SQLの各行の意味: - **create table books (...)** ... 「booksという名前の表を作る」命令。 - 各行は「**列名 + 型 + 制約**」の形。**title text not null** は「titleという列、文字列型、空は不可」。 - **primary key**（主キー）... 各行を見分けるための特別な列。本でいう「管理番号」。 - **not null** ... 「この列は必ず値を入れる」というルール（制約）。 - **default 値** ... 「省略したらこの値を入れる」初期値の指定。

▼ コードを1つずつ分解して解説

SQL を初めて見る方向けに、この **create table** 文を塊（かたまり）ごとにていねいに分解します。順番に見ていきましょう。

解説1: テーブルを作る宣言と全体の形

```
create table books (  
  -- ...列の定義がここに並ぶ...  
);
```

- `create table` (クリエイト・テーブル) は「新しいテーブル (表) を作りなさい」というSQLの命令です。 `create` = 作る、 `table` = 表。
- 続く `books` がテーブル名 (表の名前) です。ここでは「本のデータを入れる表」なので `books` としています。
- `(` から `)` までの中に、「どんな列 (縦の項目) を持つか」を1行ずつカンマ区切りで並べます。各列は基本的に「列名 型 制約」の順で書きます。
- 最後の `;` (セミコロン) は「この命令はここで終わり」という区切りです。SQLでは1つの命令の終わりに付けます。

用語: テーブル (table) とはデータを入れる「表」のこと。Excelの1枚のシートをイメージすると分かりやすいです。**列 (カラム/column)** が縦の項目 (タイトル・著者など)、**行 (ロウ/row)** が1件分のデータ (本1冊分) にあたります。

解説2: 主キーになるID列 (`primary key` と `default`)

```
id uuid primary key default gen_random_uuid(),
```

- これは `id` という列を1つ定義している行です。読み方は「列名 `id` / 型 `uuid` / 制約 `primary key` と `default ...`」の順です。
- `uuid` (ユーユアアイディー) は型で、「ランダムで重複しない長いID文字列」を入れるための型です。例: `3f2a...-...`。
- `primary key` (プライマリ・キー = 主キー) は制約で、「この列の値で各行を1件ずつ見分ける」という宣言です。本でいう「重複しない管理番号」の役割で、空 (null) も重複も許されません。
- `default gen_random_uuid()` は「値を省略したら、自動でランダムなUUIDを振る」という初期値の指定です。`gen_random_uuid()` はその場でランダムなIDを生成するPostgreSQLの関数で、`()` は「関数を呼び出す」印です。

用語: 主キー (primary key) とは「各行を一意 (ユニーク) に識別するための列」。制約 (constraint) とは「この列はこういう値しか入れてはダメ」というルールのこと、`not null` や `primary key` がその例です。

解説3: 必須の文字列列 (`text` と `not null`)

```
title text not null,  
author text not null,
```

- `title` と `author` という2つの列を定義しています。どちらも「列名 / 型 `text` / 制約 `not null`」の形です。
- `text` (テキスト) は型で、「文字列 (長さ制限のない文章)」を入れる列であることを表します。本のタイトルや著者名はこれで十分です。
- `not null` (ノット・ヌル) は制約で、「この列は空にできない = 必ず値を入れる」という意味です。`null` (ヌル = 値が無い状態) を許さない、という宣言です。
- 行末の `,` (カンマ) は「列の定義がまだ続く」という区切りです。次の列の定義へつながります。

用語: `null` (ヌル) とは「値が存在しない・未設定」を表す特別な状態のこと (空文字 `""` や `0` とは別物)。
`not null` はその `null` を禁止する制約です。

解説4: 初期値つきの列と省略可能な列 (`default` と制約なし)

```
status text not null default '未読',  
memo text,
```

- `status` 列は「`text` 型 / `not null` (必須) / `default '未読'`」という組み合わせです。`default '未読'` は「値を省略したら自動で `'未読'` を入れる」という初期値指定で、SQLでは文字列を `'...'` (シングルクォート) で囲みます。
- `not null` と `default` を両方付けると、「必ず値は入る (省略時は初期値が入るから)」という安心な列になります。
- 一方 `memo` 列は `text` だけで `not null` がありません。これは「空 (`null`) でもOK = 省略可能」という意味になります。メモは書かない本もあるためです。

用語: `default` (デフォルト) とは「値を指定しなかったときに自動で入る初期値」のこと。`default '未読'` なら、本を登録するときに状態を書かなければ自動的に「未読」になります。

解説5: 登録日時を自動で入れる列 (`timestamp` と `now()`)

```
created_at timestamp with time zone default now()
```

- `created_at` (クリエイテッド・アット = 作成日時) という列を定義しています。「いつ登録されたか」を記録するための列です。
- `timestamp with time zone` (タイムスタンプ・ウィズ・タイムゾーン) は型で、「タイムゾーン (時差) 情報付きの日時」を入れる列です。世界中どこで使っても時刻がずれない、丈夫な日時型です。
- `default now()` は「値を省略したら、その時点の現在時刻を自動で入れる」という指定です。`now()` は「今の時刻」を返すPostgreSQLの関数です。
- この行の末尾にカンマが無い点に注目してください。 `( ... )` の中の最後の列定義なので、後ろに続く列が無く、カンマを付けません。

用語: timestamp (タイムスタンプ) とは「年月日と時分秒をひとまとめた日時データ」のこと。`with time zone` を付けると時差の情報も一緒に保存され、地域をまたいでも正しい時刻として扱えます。

3.4 (任意) テストデータを入れる

動作確認用に、最初から数冊入れておくくと便利です。同じくSQL Editorで実行します。

▼ このコードがやること (先に日本語で) : さっき作った `books` テーブルに、お試し用の本のデータを3冊分まとめて登録します。`insert into` (インサート) は「表に新しい行 (データ) を追加しなさい」というSQL命令です。`(title, author, status)` で「どの列に入れるか」を指定し、`values (...)` の後ろに実際の値を並べます。1つのカッコ = 1冊分で、複数冊はカンマで区切る、と覚えてください。

```
-- insert into : テーブルにデータ (行) を追加するSQL命令
-- books (title, author, status) : どの列に値を入れるか指定
-- values (...) : 入れる値。1行が1冊分。複数行はカンマで区切る
insert into books (title, author, status) values
('リーダブルコード', 'Dustin Boswell', '読了'),
('達人プログラマー', 'David Thomas', '読書中'),
('プロを目指す人のためのTypeScript入門', '鈴木 僚太', '未読');
```

▼ コードを1つずつ分解して解説

データを追加する `insert into` 文を、塊ごとに分解します。

解説1: どのテーブルのどの列に入れるか (`insert into ... (列名)`)

```
insert into books (title, author, status) values
```


- `insert into` (インサート・イントウ) は「テーブルに新しい行 (データ) を追加しなさい」というSQLの命令です。
`insert` = 挿入する。
- 続く `books` が追加先のテーブル名です。3.3で作った `books` 表にデータを入れます。
- `(title, author, status)` は「どの列に値を入れるか」の指定です。ここに書かなかった列 (`id` や `created_at`) は、3.3で設定した `default` (自動生成・現在時刻) が働いて自動で埋まります。
- `values` (バリュース = 値) は「この後ろに実際の値を並べます」という合図です。

用語: insert (インサート) は「データを1件以上、表に新しく追加する」操作のこと。表の構造 (列) を作る `create table` とは違い、`insert` は中身のデータを足す命令です。

解説2: 実際の値を並べる (`values (...)` と複数行)

```
( 'リーダーコード', 'Dustin Boswell', '読了' ),
( '達人プログラマー', 'David Thomas', '読書中' ),
( 'プロを目指す人のためのTypeScript入門', '鈴木 僚太', '未読' );
```

- 1つのカッコ (`...`) が「1冊分 (=1行)」のデータです。中の値は、解説1で指定した `(title, author, status)` の順番どおりに並べます。1つ目が `title`、2つ目が `author`、3つ目が `status` です。
- 値が文字列なので、それぞれ `'...'` (シングルクォート) で囲みます。SQLでは文字列はシングルクォートで囲む決まりです。
- 各行は `,` (カンマ) で区切り、最後の行の末尾だけ `;` (セミコロン) で命令の終わりを示します。これで「3冊分をまとめて1回で追加」できます。

用語: values (バリュース) は「insert で実際に入れる値の並び」を指定する句のこと。`(値1, 値2, ...)` のカッコ1組が1行分にあたり、カンマで区切れば複数行を一度に追加できます。

3.5 行レベルセキュリティ (RLS) の設定

Supabaseは安全のため、初期状態では外部からデータを読み書きできないようロックされています。学習用に、まずは「誰でも読み書きできる」設定にします。

▼ このコードがやること (先に日本語で) : `books` テーブルへのアクセス制限 (鍵) を設定します。Supabaseは安全のため初期状態では外部からの読み書きを止めているので、まず `enable row level security` で「鍵の仕組み (誰がアクセスできるかのルール)」を有効にし、次に `create policy` (ポリシー = 許可ルール) で「学習用に全員に全操作を許可する」という規則を作ります。`using (true) / with check (true)` の `true` は「常に許可」という意味です。この設定は学習用で、公開アプリでは必ず制限を厳しくする点だけ押さえておきましょう。


```

-- alter table ... enable row level security : このテーブルに「行レベルセキュリティ」を有効化
alter table books enable row level security;

-- create policy : アクセスを許可する「ポリシー（規則）」を作る
-- 学習用に「全員に全操作を許可」する規則を作成（本番アプリでは後で厳しくする）
create policy "Enable all access for books"    -- ポリシーの名前（自由）
  on books                                     -- 対象テーブル
  for all                                       -- for all : 全操作（読み・書き・更新・削除）
  を対象に
  using (true)                                -- using (true) : 読み取り条件＝常に許可
  with check (true);                          -- with check (true) : 書き込み条件＝常に許可
可

```

「RLS（Row Level Security）」とは？「行（Row）ごとに、誰がアクセスできるかを制御するセキュリティ」のこと。本来は「自分のデータは自分しか見られない」のように細かく制御します。本書は学習用なので一旦「全員OK」にしますが、実際に公開するアプリでは必ず適切に制限してください（第10章で再度触れます）。

▼ コードを1つずつ分解して解説

このRLS設定は「鍵の仕組みを有効にする」命令と「許可ルールを作る」命令の2つに分かれます。塊ごとに見ていきましょう。

解説1: 鍵の仕組みを有効にする（`alter table ... enable row level security`）

```
alter table books enable row level security;
```

- `alter table`（オルター・テーブル）は「既存のテーブルの設定を変更しなさい」という命令です。`alter` = 変更する。すでにある `books` 表に対して設定を加えます。
- 続く `books` が対象のテーブル名です。
- `enable row level security`（イネーブル・ロウ・レベル・セキュリティ）は「行レベルセキュリティ（RLS）を有効にする」という指示です。これを有効にすると、ポリシー（許可ルール）が無い限り誰もアクセスできない状態になります。だから次の解説2でルールを作ります。

用語: RLS（行レベルセキュリティ）とは「行（1件のデータ）ごとにアクセスの可否を判定する仕組み」のこと。有効化しただけでは『全部禁止』なので、許可ルール（ポリシー）とセットで使います。

解説2: 許可ルールを作る (`create policy` の対象指定)

```
create policy "Enable all access for books" -- ポリシーの名前 (自由)
  on books -- 対象テーブル
  for all -- for all : 全操作 (読み・書き・更新・削除)
  to users -- 対象に
```

- `create policy` (クリエイト・ポリシー) は「アクセスを許可するルール (ポリシー) を作りなさい」という命令です。
- `"Enable all access for books"` はポリシーの名前で、自由に付けられます (後で見分けるためのラベル)。
- `on books` は「どのテーブルに対するルールか」の指定で、ここでは `books` 表が対象です。
- `for all` は「どの操作を対象にするか」の指定で、`all` = 全操作 (読み取り `select`・追加 `insert`・更新 `update`・削除 `delete`) すべてを許可対象にします。

用語: ポリシー (policy) とは「RLSにおける『誰が・どの操作を許されるか』の許可ルール」のこと。1つのテーブルに複数のポリシーを作ってもできます。

解説3: 許可する条件 (`using` と `with check`)

```
using (true) -- using (true) : 読み取り条件=常に許可
with check (true); -- with check (true) : 書き込み条件=常に許可
```

- `using (...)` は「読み取り・削除などで、どの行を対象にしてよいか」の条件です。カッコ内が `true` になる行だけが対象になります。
- `with check (...)` は「追加・更新で、どんな値を書き込んでよいか」の条件です。こちらもカッコ内が `true` になる場合だけ書き込みを許します。
- 両方とも中身が `true` (=つねに成立) なので、「すべての行・すべての書き込みを無条件に許可」になります。これが「学習用に全員OK」の正体です。末尾の `;` でこの `create policy` 命令が終わります。

用語: using / with check はポリシーの条件式。`using` は「読める・消せる行の条件」、`with check` は「書き込める内容の条件」を表します。`true` は「常に許可」、本番では `auth.uid() = user_id` のように『本人だけ許可』といった条件に書き換えます。

4. アプリからSupabaseに接続する準備

4.1 接続情報（URLとキー）を取得する

アプリからSupabaseにつながるには、2つの情報が必要です。

1. Supabaseダッシュボードの左メニュー「Project Settings（歯車アイコン）」→「API」を開きます。
2. 次の2つを控えます。

項目	何か
Project URL	あなたのデータベースの住所（ <code>https://xxxx.supabase.co</code> の形）
anon public key	アプリから接続するための公開鍵（長い文字列）

「anon public key」は公開してOK？ これは「匿名（anonymous）ユーザー用の公開鍵」で、アプリに埋め込む前提のキーです。データの保護は前述のRLS（行レベルセキュリティ）で行います。一方、`service_role` という別のキーは絶対に公開してはいけない管理者キーなので、アプリには入れないでください。

4.2 必要なライブラリをインストールする

第1章で作った `my-books-app` フォルダで、ターミナルから次を実行します。

```
npx expo install @supabase/supabase-js @react-native-async-storage/async-storage react-native-url-polyfill
# npx expo install : Expo環境に合ったバージョンで部品をインストールするコマンド (npm installのExpo版)
# @supabase/supabase-js : SupabaseをJS/TSから操作する公式ライブラリ
# @react-native-async-storage/async-storage : ログイン状態などを端末に保存する補助 (Supabaseが内部で使う)
# react-native-url-polyfill : React NativeでURL機能を補う部品 (Supabaseの動作に必要)
```

なぜ `npm install` ではなく `npx expo install` ? `npx expo install` は「今のExpoのバージョンと相性の良い版」を自動で選んでくれます。React Native関連の部品はバージョンの相性が重要なので、Expoプロジェクトでは原則こちらを使います。

4.3 環境変数を設定する（接続情報を安全に持つ）

接続情報をコードに直接書かず、環境変数（かんきょうへんすう）という外部ファイルに置きます。プロジェクト直下に `.env` というファイルを新規作成し、次を書きます。

▼このコードがやること（先に日本語で）：Supabaseへの接続情報（住所URLと公開鍵）を、コードとは別の `.env` という設定ファイルに書いて保管します。名前=値 の形で1行ずつ書くだけで、アプリから読み取れる「環境変数（外部に置く設定値）」になります。ポイントは名前を `EXPO_PUBLIC_` で始めること——Expoではこの接頭辞が付いた変数だけがアプリから読める決まりだからです。= の右側には、4.1で控えた自分の値を貼り付けます（クォートで囲む必要はありません）。

```
# .env ファイル（プロジェクトの一番上の階層に置く）
# EXPO_PUBLIC_ で始まる名前にすると、Expoアプリ内から読み取れる決まり
EXPO_PUBLIC_SUPABASE_URL=https://あなたのプロジェクト.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=あなたのanon_publicキー
# = の右側に、4.1で控えた値を貼り付ける（クォートで囲む必要はない）
```

「環境変数」と `.env` ファイルとは？「アプリの外側に置く設定値」のことです。接続情報やパスワードをコードに直書きすると、GitHubに公開したとき他人に見られて危険です。`.env` という別ファイルに分け、これをGitの管理から外すことで安全を保ちます。

`EXPO_PUBLIC_` という接頭辞の意味: Expoでは、`EXPO_PUBLIC_` で始まる環境変数だけがアプリのコードから読めます。逆にこの接頭辞が無い変数はアプリに含まれません。なお `PUBLIC`（公開）の名の通り、ここに入れた値は最終的にアプリに埋め込まれるので、`service_role` のような秘密キーは入れないこと。

`.env` をGit管理から外すため、`.gitignore` というファイル（無ければ作成）に次の1行を追加します。

```
# .gitignore ファイル（Gitに無視させたいファイルを列挙する）
.env
# ↑ この行で「.envファイルはGitで記録しない（=GitHubにアップしない）」という指定になる
```

`.gitignore` とは？「Gitに記録（追跡）させたくないファイルの一覧」を書くファイルです。`.env`（秘密情報）や `node_modules`（巨大な部品本体）などを書いておくと、GitHubにアップロードされず安全 & 軽量になります。Expoのテンプレートには最初から `.gitignore` があり、`node_modules` などは既に書かれています。

4.4 Supabaseクライアントを作る

アプリ内で何度も使う「Supabaseへの接続オブジェクト」を1か所にまとめます。プロジェクト内に `lib` フォルダを作り、その中に `supabase.ts` を新規作成します。

▼このコードがやること（先に日本語で）：アプリのあちこちから使う「Supabaseへの接続オブジェクト」を1か所で作り、共有できるようにします。`createClient(URL, キー, 設定)`という関数に、先ほど `.env` に書いた接続情報（`process.env.OO` で読み取る）を渡すと、接続オブジェクトができあがります。最後に `export` しておくことで、他のファイルから `import` して同じ接続を使い回せます。「接続の作り方を1つのファイルにまとめて、みんなで共有する」のが狙いだと押さえておきましょう。

```
// lib/supabase.ts - Supabaseへの接続を作る共通ファイル

import "react-native-url-polyfill/auto"; // RNでURL機能を有効化（おまじない。先頭で読み込む）
import AsyncStorage from "@react-native-async-storage/async-storage"; // 端末保存の部品
import { createClient } from "supabase/supabase-js"; // Supabaseクライアントを作る関数を借りる

// process.env.XXX : .envファイルに書いた環境変数を読み取る書き方
const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL!; // 末尾の!は「nullではないと保証」する印
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!;

// createClient(URL, キー, 設定) : Supabaseへの接続オブジェクトを作る
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: AsyncStorage, // ログイン状態を端末(AsyncStorage)に保存する設定
    autoRefreshToken: true, // 認証トークンを自動更新する
    persistSession: true, // セッション（ログイン状態）を保持する
    detectSessionInUrl: false, // URLからのセッション検出はモバイルでは不要なのでオフ
  },
});
// export しているので、他のファイルから import { supabase } from "../lib/supabase" で使える
```

！（エクスクラメーション）の意味: `process.env.EXPO_PUBLIC_SUPABASE_URL!` の末尾の `!` は、TypeScriptに「この値はnull/undefinedではないと約束する」と伝える印です。環境変数は型の上では「無いかもしれない」扱いなので、`!` で「ちゃんと設定してあるから大丈夫」と明示しています。これを「非null表明（ひ・ヌル・ひょうめい、non-null assertion）」と呼びます。第2章で出た「型アサーション `as` 」と同じく、「型の上での扱いを、開発者が責任を持って言い換える」書き方の一種です。

「おまじない」と書いた `react-native-url-polyfill/auto` の正体: コードの1行目で読み込んでいるこの部品には、ちゃんと役割があります。SupabaseのライブラリはWebブラウザに最初から備わっている `URL` という機能（住所文字列を分解・組み立てする道具）を使います。ところがReact Native（スマホアプリの実行環境）には、この `URL` 機能が標準で備わっていません。そのままだとSupabaseが内部で動けずエラーになります。`react-native-url-polyfill/auto` は、その足りない `URL` 機能を補って追加する部品です。

「ポリフィル（polyfill）」とは？「ある環境に足りない機能を、後から埋めて補う部品」のこと。壁の穴を埋めるパテ（filler）が語源です。「RNに無いブラウザ機能を埋めるもの」と覚えてください。必ず他の処理より先に読み込む必要があるため、ファイルの先頭に書きます。

`process.env` と `EXPO_PUBLIC_` のしくみ（なぜこの接頭辞が必要か）: `process.env.〇〇` は「`.env` ファイルに書いた環境変数を読み取る」書き方です（第4.3節で `.env` を作りました）。ただしExpoには大事なルールがあります—— `EXPO_PUBLIC_` で始まる名前の変数だけが、アプリのコード（`process.env`）から読めます。なぜそんなルールがあるかというと、安全のためです。`.env` には秘密の値（管理者キーなど）を入れることもあります。もし全部の変数がアプリに埋め込まれてしまうと、秘密が漏れる危険があります。そこでExpoは「`EXPO_PUBLIC_`（＝「公開してよい」という意思表示）を付けた変数だけアプリに含める」という線引きをしています。逆に言うと、ここに入れた値は最終的にアプリに埋め込まれ、利用者から見られ得るので、`anon`（公開前提）キーはOKでも、`service_role`（管理者）キーのような秘密は絶対に入れないでください（第4.1節の注意の再確認です）。

▼ コードを1つずつ分解して解説

このファイルを「部品を借りる」「接続情報を読む」「接続を作って配る」の3つの塊に分けて見ていきます。

解説1: 必要な部品を借りる（`import`）

```
import "react-native-url-polyfill/auto"; // RNでURL機能を有効化（おまじない。  
先頭で読み込む）  
import AsyncStorage from "@react-native-async-storage/async-storage"; // 端末保存の部品  
import { createClient } from "@supabase/supabase-js"; // Supabaseクライアントを作る関数を借りる
```

- `import`（インポート）は「他のファイルやライブラリの機能を借りてくる」命令です。このファイルの先頭に並べます。
- 1行目の `import "...";` は「名前を付けずにただ読み込むだけ」の書き方です。`react-native-url-polyfill/auto` を読み込むと、足りない `URL` 機能が自動で補われます（前述のポリフィル）。最初に読み込む必要があるため先頭に置きます。

- 2行目 `import AsyncStorage from "..."` は「そのライブラリの主役（デフォルトの機能）を `AsyncStorage` という名前で借りる」書き方です。
- 3行目 `import { createClient } from "..."` は「ライブラリの中から `createClient` という関数だけを名指しで借りる」書き方です。`{ }` で囲むのがその目印です。

用語: `import` は他ファイルの機能を取り込む文。`from` の右側がどこから借りるか、左側がどの名前で受け取るか。`{ 名前 }` 付きは「名前指定の借用（named import）」、付かないものは「主役の借用（default import）」です。

解説2: 接続情報を読み取る（`process.env` と `!`）

```
const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL!;           // 末尾の ! は「nullではないと保証」する印
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!;
```

- `const 名前 = 値;` は「変えない変数を1つ作る」書き方です。ここでは接続情報を変数に入れて、後で使いやすくしています。
- `process.env.〇〇` は「`.env` ファイルに書いた環境変数を読み取る」書き方です。4.3で書いた `EXPO_PUBLIC_SUPABASE_URL` などの値がここに入ります。
- 末尾の `!`（エクスクラメーション）は「この値はnull/undefinedではないと保証する」印（非null表明）です。環境変数は型の上では「無いかもしれない」扱いなので、`!` で「ちゃんと設定済みだから大丈夫」と明示します。

用語: 環境変数（environment variable）は、コードの外側（`.env` ファイルなど）に置く設定値のこと。`process.env.名前` で読み出します。非null表明 `!` は「nullではないと開発者が責任を持って言い切る」TypeScriptの書き方です。

解説3: 接続オブジェクトを作って共有する（`createClient` と `export`）

```
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: AsyncStorage,           // ログイン状態を端末(AsyncStorage)に保存する設定
    autoRefreshToken: true,          // 認証トークンを自動更新する
    persistSession: true,            // セッション（ログイン状態）を保持する
    detectSessionInUrl: false,       // URLからのセッション検出はモバイルでは不要なのでオフ
  },
});
```

- `createClient(URL, キー, 設定)` は「Supabaseへの接続オブジェクトを作る」関数です。第1引数に住所(URL)、第2引数に公開鍵、第3引数に動作の設定を渡します。

- 第3引数の `{ auth: { ... } }` は設定オブジェクトです。 `auth`（認証）の中で、ログイン状態の保存先や自動更新などを細かく指定しています。各設定は「設定名: 値」の形で、 `true` / `false` や保存先の部品を渡します。
- `storage: AsyncStorage` は「ログイン状態を端末に保存する」設定で、解説1で借りた `AsyncStorage` をここで使っています。
- 先頭の `export`（エクスポート）は「この `supabase` を他のファイルからも使えるように外へ公開する」印です。これにより他の画面から `import { supabase } from "../lib/supabase"` で同じ接続を使い回せます。

用語: `export` は `import` の逆で、「このファイルの機能を外部に提供する」宣言。 `export const supabase` とすれば、アプリ中のどのファイルからも同じ接続オブジェクトを借りて使えます。

4.5 接続テスト

きちんとつながるか確認しましょう。任意の画面ファイル（例: `app/(tabs)/index.tsx`）に、一時的に次を足して確認します。

▼ このコードがやること（先に日本語で）：作ったSupabase接続が本当に動くかを、実際にデータを取得して確かめます。画面が表示されたときに1回だけ（ `useEffect` ） `books` テーブルから全データを読み取り、その結果をターミナルに表示します。 `supabase.from("books").select("*")` が「booksテーブルから全列を取ってきて」という命令で、 `await` で結果を待ちます。成功すれば本のデータが、失敗すればエラー内容がログに出る——という動作確認専用のコードで、確認できたら消します。

```
import { useEffect } from "react";
import { supabase } from "../../lib/supabase"; // 作った接続オブジェクトを借りる（パスは場所に応じて調整）

export default function HomeScreen() {
  // useEffect : 画面表示時に1回だけ実行（第3章参照）
  useEffect(() => {
    // 即時実行する非同期関数（async/awaitを使うため）。第2章のasync/await参照
    const test = async () => {
      // supabase.from("books").select("*") : booksテーブルから全列(*)を取得する
      // await : 結果が返るまで待つ / { data, error } : 結果を分割代入で受け取る
      const { data, error } = await supabase.from("books").select("*");
      if (error) {
        console.log("接続エラー:", error.message); // 失敗時はエラー内容を表示
      } else {
        console.log("取得した本:", data); // 成功時はデータを表示
      }
    };
    test();
  }, []);

  // （画面の見た目は省略。ターミナルのログで結果を確認する）
}
```

確認方法: アプリを起動（`npx expo start`）してこの画面を開き、**ターミナル**を見ます。「取得した本: [...]」と3冊分のデータ（3.4でテストデータを入れた場合）が表示されれば、接続成功です！「接続エラー」が出たら、`.env`の値が正しいか、RLSのポリシー（3.5）を設定したかを確認してください。

確認できたら消す: このテストコードは確認用です。動作確認できたら消して、次章からの本実装に進みます。

▼ コードを1つずつ分解して解説

接続テストのコードを「部品を借りる」「表示時に1回だけ実行」「データを取って結果を出す」の塊に分けて見ます。

解説1: 必要な部品を借りる（`import`）

```
import { useEffect } from "react";
import { supabase } from "../../lib/supabase"; // 作った接続オブジェクトを借りる（パスは場所に応じて調整）
```

- `import { useEffect } from "react"` で、Reactから `useEffect`（画面表示時などに処理を動かす道具。第3章参照）を名指しで借りています。

- `import { supabase } from "../../lib/supabase"` で、4.4で作って `export` した接続オブジェクト `supabase` を借りています。 `../../..` は「2つ上の階層のフォルダ」を意味し、ファイルの置き場所に応じて調整します。

用語: 相対パスとは「今のファイルから見た位置」での指定方法。 `./` が同じ階層、 `../` が1つ上の階層、 `../../` が2つ上の階層を表します。

解説2: 画面表示時に1回だけ実行する (`useEffect`)

```
useEffect(() => {
  // ...中の処理...
}, []);
```

- `useEffect(処理, 依存配列)` は「画面が表示された後などのタイミングで、指定した処理を動かす」ためのReactの道具です（詳しくは第3章）。
- 第1引数の `() => { ... }` が「実行したい処理」です。
- 第2引数の `[]`（空の依存配列）が重要で、「最初に画面が表示されたときに1回だけ実行する」という意味になります。空でないと毎回の更新で動いてしまいます。

用語: 依存配列 (dependency array) とは `useEffect` の第2引数の `[ ... ]` のこと。中に入れた値が変わるたびに処理が再実行されます。空 `[]` なら「初回の1回だけ」になります。

解説3: データを取得して結果を出す (`async/await` と `supabase.from`)

```
const test = async () => {
  const { data, error } = await supabase.from("books").select("*");
  if (error) {
    console.log("接続エラー:", error.message); // 失敗時はエラー内容を表示
  } else {
    console.log("取得した本:", data); // 成功時はデータを表示
  }
};
test();
```

- `const test = async () => { ... }` は「`async`（非同期）な関数 `test` を作る」書き方です。`async` を付けると、中で `await`（結果を待つ）が使えます（第2章参照）。
- `supabase.from("books").select("*")` がデータ取得の命令です。`from("books")` で「booksテーブルから」、`select("*")` で「全列（* = すべて）を取ってくる」という意味になります。
- `await` は「結果が返ってくるまで待つ」印です。通信は時間がかかるため、待ってから結果を受け取ります。

- `const { data, error } = ...` は分割代入で、結果から `data`（取れたデータ）と `error`（エラー情報）を同時に取り出します。`if (error)` でエラーの有無を判定し、`console.log(...)` でターミナルに結果を表示します。
- 最後の `test();` で、作った関数を実際に呼び出して実行します（定義しただけでは動かないため）。

用語: `async/await` は「時間のかかる処理（通信など）の結果を待ってから次に進む」ための書き方。
`console.log(...)` は「ターミナル（コンソール）に値を出力して確認する」ためのデバッグ用命令です。

5. この章のまとめ

- データを永続化するには**端末内**か**クラウド**の保存先が必要。本書は複数端末同期できる**クラウド型**のSupabaseを採用
- 保存方法の選択肢（Supabase / Firebase / SQLite / AsyncStorage）を比較し、目的別の選び方を理解した
- Supabaseでプロジェクトと **books** テーブルをSQLで作成し、学習用にRLS（行レベルセキュリティ）を設定した
- 接続情報を `.env`（環境変数）に安全に保管し、`.gitignore` でGit管理から除外した
- `lib/supabase.ts` にSupabaseクライアントを作り、接続テストに成功した

次の章へ: データの保管庫が整いました。第6章では、アプリの画面構成（ナビゲーション）を本格的に組み立て、第7・8章でいよいよCRUD（一覧・作成・編集・削除）を実装します。